

Explicit Modeling and Solving of an (MI)LP with JuMP

The optimization problem

$$\left[\begin{array}{rclcl} \max z(x) = & x_1 & + & 3x_2 & (0) \\ s.t & x_1 & + & x_2 & \leq 14 \quad (1) \\ & -2x_1 & + & 3x_2 & \leq 12 \quad (2) \\ & 2x_1 & - & x_2 & \leq 12 \quad (3) \\ & x_1 & , & x_2 & \geq 0 \quad (4) \end{array} \right]$$

Declaring the packages to use



```
julia> using JuMP, HiGHS
```



Creating a model:

```
modelId = Model( )
```

```
julia> modLP = Model( )
```

Result:

[]

```
@variable(modelId, variable)
```

```
julia> @variable(modLP, x1 >= 0)
julia> @variable(modLP, x2 >= 0)
```

$$\left[\begin{array}{c} x_1 \\ x_2 \end{array} \right] \geq 0 \quad (4)$$



Defining an objective function:

```
@objective(modelId, sense, expression)
```

```
julia> @objective(modLP, Max, x1 + 3x2)
```

Result:

$$\left[\begin{array}{rcll} \max z(x) = & x_1 & + & 3x_2 & (0) \\ & x_1 & , & x_2 & \geq 0 & (4) \end{array} \right]$$



Defining a constraint:

```
@constraint(modelId, id, constraint)
```

```
julia> @constraint(modLP, cst1, x1 + x2 <= 14)
julia> @constraint(modLP, cst2, -2x1 + 3x2 <= 12)
julia> @constraint(modLP, cst3, 2x1 - x2 <= 12)
```

Result:

$$\left[\begin{array}{rcllcl} \max z(x) = & x_1 & + & 3x_2 & & (0) \\ s.t & x_1 & + & x_2 & \leq & 14 & (1) \\ & -2x_1 & + & 3x_2 & \leq & 12 & (2) \\ & 2x_1 & - & x_2 & \leq & 12 & (3) \\ & x_1 & , & x_2 & \geq & 0 & (4) \end{array} \right]$$

Solving the optimization problem



```
set_optimizer(modelId, solver )
```

```
julia> set_optimizer(modLP, HiGHS.Optimizer)
```

```
set_silent(modelId )
```

```
julia> set_silent(modLP)
```

```
optimize!(modelId)
```

```
julia> optimize!(modLP)
```


Solving the optimization problem



```
set_optimizer(modelId, solver )
```

```
julia> set_optimizer(modLP, HiGHS.Optimizer)
```

```
set_silent(modelId )
```

```
julia> set_silent(modLP)
```

```
optimize!(modelId)
```

```
julia> optimize!(modLP)
```

Solving the optimization problem



```
set_optimizer(modelId, solver )
```

```
julia> set_optimizer(modLP, HiGHS.Optimizer)
```

```
set_silent(modelId )
```

```
julia> set_silent(modLP)
```

```
optimize!(modelId)
```

```
julia> optimize!(modLP)
```



```
julia> @show is_solved_and_feasible(modLP)
```

```
julia> @show termination_status(modLP)
```

```
julia> @show objective_value(modLP)
```

```
julia> @show value(x1)
```

```
julia> @show value(x2)
```



```
julia> @show is_solved_and_feasible(modLP)
```

```
julia> @show termination_status(modLP)
```

```
julia> @show objective_value(modLP)
```

```
julia> @show value(x1)
```

```
julia> @show value(x2)
```



```
julia> @show is_solved_and_feasible(modLP)
```

```
julia> @show termination_status(modLP)
```

```
julia> @show objective_value(modLP)
```

```
julia> @show value(x1)
```

```
julia> @show value(x2)
```

Summary (explicit model)



```
julia> using JuMP, HiGHS

julia> modLP = Model( )
julia> @variable(modLP, x1 >= 0)
julia> @variable(modLP, x2 >= 0)
julia> @objective(modLP, Max, x1 + 3x2)
julia> @constraint(modLP, cst1, x1 + x2 <= 14)
julia> @constraint(modLP, cst2, -2x1 + 3x2 <= 12)
julia> @constraint(modLP, cst3, 2x1 - x2 <= 12)

julia> set_optimizer(modLP, HiGHS.Optimizer)
julia> set_silent(modLP)
julia> optimize!(modLP)

julia> @show is_solved_and_feasible(modLP)
julia> @show objective_value(modLP)
julia> @show value(x1), value(x2)
```

↳ https://github.com/xgandibleux/EURO_MCDM2026/blob/main/3.examples/1LPexplicit.ipynb

Details on

Variables:

- ▶ by default, the variables are **continuous** and **unbounded**:

```
julia> @variable(model, x)           # x is free
```

- ▶ possible to setup lower and/or upper bounds on a variable:

```
julia> @variable(model, x ≥ lb))    # x is bounded  
julia> @variable(model, x ≤ ub)  
julia> @variable(model, lb ≤ x ≤ ub)  
julia> @variable(model, x == 2)     # x is fixed
```

- ▶ possible to specify the type of a variable:

```
julia> @variable(model, x ≥ 0, Int)    #  $x \in \mathbb{N}$   
julia> @variable(model, x, Bin)       #  $x \in \{0, 1\}$ 
```

Details on

Variables:

- ▶ by default, the variables are **continuous** and **unbounded**:

```
julia> @variable(model, x)           # x is free
```

- ▶ possible to setup lower and/or upper bounds on a variable:

```
julia> @variable(model, x ≥ lb))    # x is bounded  
julia> @variable(model, x ≤ ub)  
julia> @variable(model, lb ≤ x ≤ ub)  
julia> @variable(model, x == 2)     # x is fixed
```

- ▶ possible to specify the type of a variable:

```
julia> @variable(model, x ≥ 0, Int)    #  $x \in \mathbb{N}$   
julia> @variable(model, x, Bin)       #  $x \in \{0, 1\}$ 
```


Details on

Variables:

- ▶ by default, the variables are **continuous** and **unbounded**:

```
julia> @variable(model, x)           # x is free
```

- ▶ possible to setup lower and/or upper bounds on a variable:

```
julia> @variable(model, x ≥ lb))    # x is bounded  
julia> @variable(model, x ≤ ub)  
julia> @variable(model, lb ≤ x ≤ ub)  
julia> @variable(model, x == 2)     # x is fixed
```

- ▶ possible to specify the type of a variable:

```
julia> @variable(model, x ≥ 0, Int)    #  $x \in \mathbb{N}$   
julia> @variable(model, x, Bin)       #  $x \in \{0, 1\}$ 
```